CloudCP Report Testing — Live Execution Plan

Scope: `CloudCpReportTesting/` (writes only here) **Reference reads:** `CloudCpCliTesting/bryckclient-cli/`, `dataset_cloudcp/spec_files/` **Owner:** QA **Status:** Draft — awaiting sign-off before implementation

---

1. Context & Motivation

The existing synthetic cases (P4-01 → P4-08) proved the reference merge-join engine in `report_engine.py` is correct against fixture data. This plan replaces those synthetic cases with **live end-to-end test cases** that exercise the full real flow:

```
datagen → upload/download transfer → report ZIP download → parse → verify
```

This catches bugs the reference engine cannot catch: incorrect S3 paths in the real report, wrong file sizes, ETag mismatches, transfer summary counter drift, and path encoding issues in real filenames.

---

2. Assumptions

Assumption	Value
Datagen binary	`/home/bryck/rperiyas/datagen --spec <file>.yaml` — run via SSH on the Bryck
SSH access	Via `ssh_runner.SshRunner`, credentials from `login.json`
Local source root on Bryck	`/bryck/report_testing/`
S3 destination prefix	`s3://vijay/report_testing/`
Report ZIP location (after transfer)	`/opt/bryck/bryckapi/downloads/cloud_transfer_logs/cloud_transfer_<id>.zip`
Report download method	`bryck_cloud_transfer_report.py` (GET `/api/download?name=cloud_log&type=<id>`)
S3 cleanup command	`aws s3 rm --recursive s3://vijay/report_testing/<case_id> --endpoint-url https://10.10.10.103:9000`

Assumption	Value
Local cleanup	SSH: `rm -rf /bryck/report_testing/<case_id>`
Python deps on runner	`requests`, `paramiko` (same as existing bryckclient-cli)

3. Directory Structure After Implementation

```
CloudCpReportTesting/
├── run_report_tests.py          # NEW: main live-test runner (--all/--from/--to/--one/--manual)
├── report_engine.py             # EXTENDED: live report parser added alongside existing engine
├── config.json                  # NEW: polling interval, timeout, credentials path, S3 endpoint
├── spec_files/                  # NEW: datagen YAML specs, one sub-folder per case
│   ├── RT-01/
│   │   └── RT-01_small_flat.yaml
│   ├── RT-02/
│   │   └── RT-02_mixed_tiers.yaml
│   ├── RT-03/
│   │   └── RT-03_filename_variants.yaml
│   ├── RT-04/
│   │   └── RT-04_nested_dirs.yaml
│   ├── RT-05/
│   │   └── RT-05_zero_byte.yaml
│   ├── RT-06/
│   │   └── RT-06_large_single.yaml
│   └── RT-07/
│       └── RT-07_high_count.yaml
├── bryckclient-cli/             # NEW: self-contained copy of CLI tools
│   ├── bryck_api.py
│   ├── session.py
│   ├── ssh_runner.py
│   ├── bryck_cloud_transfer_initiate.py
│   ├── bryck_cloud_transfer_status.py
│   ├── bryck_cloud_transfer_report.py
│   ├── login.json               # pre-filled from existing
│   └── cloud_ops.json           # pre-filled template (user edits bryck_src/cloud_bucket/bryck_dst)
├── cases/                       # REPLACED: live case plugins (same plugin interface as before)
│   ├── __init__.py
│   ├── rt_01_small_flat.py
│   ├── rt_02_mixed_tiers.py
│   ├── rt_03_filename_variants.py
│   ├── rt_04_nested_dirs.py
│   ├── rt_05_zero_byte.py
```

```
│   ├── rt_06_large_single.py
│   ├── rt_07_high_count.py
│   ├── rt_08_reupload.py
│   ├── rt_09_download.py
│   └── rt_10_round_trip.py
└── reports/                    # timestamped output per run (unchanged)
```

---

4. Configuration — `config.json`

```
{
  "login_json": "bryckclient-cli/login.json",
  "cloud_ops_json": "bryckclient-cli/cloud_ops.json",
  "datagen_binary": "/home/bryck/rperiyas/datagen",
  "bryck_local_root": "/bryck/report_testing",
  "s3_bucket_prefix": "s3://vijay/report_testing",
  "s3_endpoint_url": "https://10.10.10.103:9000",
  "transfer_poll_interval_sec": 15,
  "transfer_poll_timeout_sec": 3600,
  "report_download_dir": "reports/zips",
  "cleanup_local": true,
  "cleanup_s3": true
}
```

All fields overridable from the CLI via `--config alternate.json`.

---

5. Full Execution Flow (Per Case)

```
┌──────────────────────────────────────────────────────────────────┐
│ 1. SETUP                                                         │
│    SSH rm -rf /bryck/report_testing/<case_id>   (idempotent reset)  │
│    S3 rm --recursive s3://vijay/report_testing/<case_id>         │
├──────────────────────────────────────────────────────────────────┤
│ 2. DATA GENERATION                                               │
│    SSH: datagen --spec spec_files/<case_id>/<spec>.yaml          │
│    Verify: SSH ls -la /bryck/report_testing/<case_id> shows files   │
│    Record: expected_file_count, expected_total_bytes from datagen   │
├──────────────────────────────────────────────────────────────────┤
│ 3. INITIATE TRANSFER (UPLOAD or DOWNLOAD)                        │
│    POST /api/bcloud/transfer (via bryck_cloud_transfer_initiate.py) │
│    Capture transfer_id from response                             │
├──────────────────────────────────────────────────────────────────┤
│ 4. POLL UNTIL TERMINAL STATE                                     │
│    GET /api/bcloud/status_transfer?id=<transfer_id>              │
│    Every poll_interval_sec seconds                               │
│    Terminal: COMPLETED, FAILED, STOPPED, CANCELLED               │
│    Timeout: poll_timeout_sec → mark case TIMEOUT (not FAIL)      │
├──────────────────────────────────────────────────────────────────┤
```

```
┌──────────────────────────────────────────────────────────────────┐
│ 5. DOWNLOAD REPORT ZIP                                           │
│    GET /api/download?name=cloud_log&type=<transfer_id>           │
│    Save to reports/zips/cloud_transfer_<id>.zip                  │
│    Unzip to reports/run_<ts>/<case_id>/cloud_transfer_<id>/      │
├──────────────────────────────────────────────────────────────────┤
│ 6. PARSE REPORT                                                  │
│    Parse transfer_report_<id>.csv  → per-file rows               │
│    Parse transfer_summary.txt      → counter block               │
│    Parse final_report.json         → S3 path + ETag snapshot (opt)  │
├──────────────────────────────────────────────────────────────────┤
│ 7. ASSERT (case-specific, see §7)                                │
│    Run case assertions against parsed data                       │
│    Write per-case README.txt + results.json                      │
├──────────────────────────────────────────────────────────────────┤
│ 8. CLEANUP                                                       │
│    SSH rm -rf /bryck/report_testing/<case_id>                    │
│    aws s3 rm --recursive s3://vijay/report_testing/<case_id>     │
│    Verify both are empty (cleanup assertion)                     │
└──────────────────────────────────────────────────────────────────┘
```

---

6. Report Files — Expected ZIP Contents

After unzip, `cloud_transfer_<id>/` contains:

File	Used for
`transfer_report_<id>.csv`	Per-file: `local_path, s3path, size, etag, status, attempt, finished_at`
`transfer_summary.txt`	Counter block: files count, bytes, missing count, transfer status
`cloud_transfer_<id>.log`	Debug log — not parsed, kept for failure triage
`cloud_transfer_txhistory_<id>.log`	Retry history — used for last-status-wins check
`final_report.json`	Optional S3 snapshot: `AbsoluteFilePath, S3Path, FileSize, ETag`

---

7. Test Cases

Naming Convention

`RT-NN` — Report Testing, numbered sequentially. Source: `/bryck/report_testing/RT-NN/` S3 dest: `s3://vijay/report_testing/RT-NN/`

RT-01 — Happy Path: Small Flat Files (Upload)

Field	Value
Spec	`spec_files/RT-01/RT-01_small_flat.yaml`

Field	Value
Files	120 files, flat dir, 1 MB – 20 MB each
Total size	~1 GB
Transfer mode	Upload

Assertions:

1. Transfer reaches `COMPLETED` state (not FAILED/STOPPED).
2. `transfer_report_<id>.csv` row count == 120.
3. Every row has `status == SUCCESS`.
4. Every `size` in CSV matches the file's actual size on `/bryck` (SSH `stat`).
5. No file in the source dir is absent from the report (`MISSING` count == 0).
6. `transfer_summary.txt` → `Number of files == 120`, `Number of missing files/objects == 0`.
7. `transfer_summary.txt` → `Transfer status == Completed`.

Edge validation:

- ETag field is non-empty for all rows.
- `finished_at` timestamps are within `[transfer_start, transfer_completion]` from summary.

RT-02 — Mixed Tier Upload

Field	Value
Spec	`spec_files/RT-02/RT-02_mixed_tiers.yaml`
Files	250 files: tiny (≤1 MB, 80 files) + small (1-100 MB, 120 files) + medium (100 MB-1 GB, 50 files)
Total size	~8 GB
Transfer mode	Upload

Assertions (all of RT-01's, plus):

1. Files from all three tiers appear in the report — no tier silently skipped.
2. The `transfer_summary.txt` byte total matches the sum of `size` across all CSV rows.
3. Bytes in summary Source Summary == Bytes in Destination Summary.

RT-03 — Filename Encoding Variants (Upload)

Field	Value
Spec	`spec_files/RT-03/RT-03_filename_variants.yaml`
Files	60 files with varied naming: spaces, parentheses, brackets, commas, periods in names, and a sub-set of unicode chars (basic CJK if datagen supports)

Field	Value
Total size	~300 MB
Transfer mode	Upload

Assertions (all of RT-01's, plus):

1. Every `local_path` in the report round-trips without truncation (parse the CSV back and check the path is identical to the source path).
2. Embedded commas and quotes in filenames do not corrupt adjacent CSV columns.
3. S3 path uses URL-safe percent-encoding where necessary (non-ASCII) but the decoded path == original filename.
4. Zero rows with `status != SUCCESS` caused by a path encoding mismatch.

RT-04 — Nested Directory Tree (Upload)

Field	Value
Spec	`spec_files/RT-04/RT-04_nested_dirs.yaml`
Files	100 files spread across 5 levels of subdirectories
Total size	~500 MB
Transfer mode	Upload

Assertions (all of RT-01's, plus):

1. The full relative path from the source root appears in `local_path` — intermediate dirs not collapsed.
2. S3 path mirrors the directory structure: `s3://vijay/report_testing/RT-04/<subdir1>/<subdir2>/.../<file>`.
3. No files from a deeper subdirectory are merged into a parent level in the report.

RT-05 — Zero-Byte Files (Upload)

Field	Value
Spec	`spec_files/RT-05/RT-05_zero_byte.yaml`
Files	10 zero-byte files (datagen `size: 0`)
Total size	0 bytes
Transfer mode	Upload

Assertions:

1. Transfer reaches `COMPLETED`.
2. All 10 files appear in the report with `status == SUCCESS`.
3. `size == 0` in every CSV row.

4. ETag is present (S3 ETag for empty object is `"d41d8cd98f00b204e9800998ecf8427e"`).
5. `transfer_summary.txt` Total size == `0 bytes`.

Known edge: If the tool silently skips zero-byte files and the count in summary is 0, that is a defect — this assertion catches it.

RT-06 — Single Large File (Upload)

Field	Value
Spec	`spec_files/RT-06/RT-06_large_single.yaml`
Files	1 file, 600 MB – 1 GB
Transfer mode	Upload

Assertions:

1. Transfer reaches `COMPLETED`.
2. Report has exactly 1 row, `status == SUCCESS`.
3. `size` in report matches the actual file size (SSH `stat`).
4. ETag is present (for large files this is a multipart ETag — non-empty is sufficient).
5. `transfer_summary.txt` file count == 1.

RT-07 — High File Count (Upload)

Field	Value
Spec	`spec_files/RT-07/RT-07_high_count.yaml`
Files	1,000 files, tiny (512 KB each), flat
Total size	~512 MB
Transfer mode	Upload

Assertions (all of RT-01's, plus):

1. Report row count == 1,000 (no files silently dropped at scale).
2. No duplicate `local_path` entries in the report.
3. `transfer_summary.txt` count == 1,000.
4. Parse time for 1,000 rows is recorded (not a blocking assertion, logged for baseline).

RT-08 — Re-Upload to Same Destination (Idempotency)

Field	Value
Spec	Same as RT-01 (reuse)
Source	`/bryck/report_testing/RT-08/` (fresh datagen run)

Field	Value
S3 dest	`s3://vijay/report_testing/RT-08/` (first pre-populate via an initial upload, then upload again)
Transfer mode	Upload × 2

Precondition: Run a first upload, confirm COMPLETED, then run a second upload to the same S3 dest without cleaning S3 first.

Assertions (second transfer):

1. Second transfer reaches `COMPLETED`.
2. All rows in the second report have `status == SUCCESS` or `SKIPPED` (depending on implementation).
3. No row has `status == FAILED` due to a "file already exists" error.
4. File counts in second summary match source counts.

RT-09 — Download Transfer (S3 → /bryck)

Field	Value
Precondition	RT-01 must have run and S3 dest is populated
S3 source	`s3://vijay/report_testing/RT-01/`
Local dest	`/bryck/report_testing/RT-09-download/`
Transfer mode	Download

Assertions:

1. Transfer reaches `COMPLETED`.
2. `transfer_report_<id>.csv` row count == 120 (same as RT-01 source count).
3. Every row has `status == SUCCESS`.
4. `size` in download report matches the size recorded in RT-01's upload report.
5. Downloaded files exist on the Bryck (SSH `ls -la`) with correct sizes.

RT-10 — Round-Trip: Upload + Download

Field	Value
Spec	`spec_files/RT-01/RT-01_small_flat.yaml` (reused)
Source	`/bryck/report_testing/RT-10-src/`
S3 intermediate	`s3://vijay/report_testing/RT-10/`
Download dest	`/bryck/report_testing/RT-10-dst/`
Transfer mode	Upload then Download

Assertions:

1. Upload transfer: all SUCCESS, count matches.
2. Download transfer: all SUCCESS, count matches upload count.
3. Cross-check: `size` of each file in download report == `size` in upload report (same ETag when possible).
4. Files at `/bryck/report_testing/RT-10-dst/` match files at `/bryck/report_testing/RT-10-src/` by name and size.

---

8. Cross-Cutting Validations (Applied to Every Case)

These checks run automatically on every test case's parsed report, regardless of which case-specific assertions pass or fail:

Check	Logic
No duplicate paths	`len(set(local_path))` == `len(all_rows)`
No null local_path	Every row has a non-empty `local_path`
No null s3path	Every row has a non-empty `s3path`
No null size	Every `size` is a valid integer ≥ 0
Transfer status is Completed	`transfer_summary.txt` → `Transfer status: Completed`
Summary count == report row count	`summary.number_of_files == len(report_rows)`
Summary missing == 0	`summary.missing_files_objects == 0`
Cleanup verified	After cleanup: SSH confirms dir gone; S3 list confirms prefix empty

---

9. CLI Interface for `run_report_tests.py`

```
run_report_tests.py --all                          # run all RT-xx cases in order
run_report_tests.py --from RT-01 --to RT-05        # run a contiguous range
(inclusive)
run_report_tests.py --one RT-03                    # run exactly one case
run_report_tests.py --manual RT-03                 # print steps and wait for manual
confirmation at each step
run_report_tests.py --list                         # list all discovered cases and
exit
run_report_tests.py --dry-run --all                # print what would run without
doing anything
run_report_tests.py --config alt.json --all        # use alternate config file
run_report_tests.py --no-cleanup --one RT-01       # skip cleanup (leave data and S3
objects for inspection)
run_report_tests.py --no-datagen --one RT-01       # skip datagen (data assumed
already present)
run_report_tests.py --no-transfer --one RT-01      # skip transfer (use --transfer-
```

```
id to supply an existing one)
run_report_tests.py --transfer-id 89 --one RT-01 # parse + verify a specific
already-completed transfer
```

Exit codes: 0 = all selected cases passed; 1 = at least one failed; 2 = configuration/setup error.

---

10. Manual Mode (`--manual`)

Prints each step with a prompt before executing it. Useful for debugging or running on a locked-down environment where automated SSH is not allowed:

```
[RT-03  Step 1/8] Cleanup: remove /bryck/report_testing/RT-03 and
s3://vijay/report_testing/RT-03
  Press Enter to execute, 's' to skip, 'q' to quit:
```

After a manual step the user is expected to confirm completion (or provide the transfer ID if step 3 — initiate transfer — was done manually).

---

11. Output Structure

```
reports/
└── run_20260819_143022/
    ├── results.json            # consolidated: all case results, pass/fail,
details
    ├── RT-01/
    │   ├── cloud_transfer_<id>/
    │   │   ├── transfer_report_<id>.csv
    │   │   ├── transfer_summary.txt
    │   │   ├── final_report.json
    │   │   └── cloud_transfer_<id>.log
    │   ├── assertions.json     # per-case assertion results
    │   └── README.txt          # human-readable case summary
    ├── RT-02/
    ...
```

---

12. Cleanup Assertions

After `cleanup_local` and `cleanup_s3` both run, the runner asserts:

1. **Local:** SSH `ls /bryck/report_testing/<case_id>` → exit code 2 (not found) or empty dir.
2. **S3:** `aws s3 ls s3://vijay/report_testing/<case_id>/ --endpoint-url ...` → zero objects listed.

If either cleanup assertion fails, the case is marked `PASS_WITH_CLEANUP_FAILURE` (not a hard FAIL since the data is just leftover, but it is surfaced in the report so it can be manually cleared).

---

13. Edge Cases & Negative Scenarios

Scenario	How Handled
Datagen fails (SSH error or non-zero exit)	Case marked `SETUP_ERROR`, skip transfer, move to next case
Transfer fails (`FAILED`/`STOPPED` state)	Attempt to download partial report; mark case `TRANSFER_FAILED`; log last error from `.log`
Transfer times out (poll_timeout_sec exceeded)	Mark case `TIMEOUT`; do not assert; attempt cleanup
Report ZIP download fails (HTTP error)	Mark case `REPORT_DOWNLOAD_ERROR`; keep partial output
ZIP is corrupt / missing expected files	Mark case `REPORT_PARSE_ERROR` with details of what was missing
S3 cleanup returns non-zero (permissions, endpoint unreachable)	Log warning; do not fail the test case; flag `CLEANUP_FAILURE`
Zero files in report (empty CSV)	Catches a complete transfer engine failure; assert: `row_count > 0`

---

14. Spec File Design (datagen YAMLs)

All spec files use:

- `root: /bryck/report_testing/<case_id>/` (avoids collision with other test phases)
- `threads: 8` (safe for dev/test environments)
- `mode: flat` for flat cases, `mode: tree` for nested-dir cases
- `content.type: random`, `direct_io.enabled: false`, `fsync: false`

Example snippet for RT-01:

```
version: 1
mode: flat
root: /bryck/report_testing/RT-01
threads: 8
seed: 1001

content:
  type: random
  buffer_size: 4MB
  direct_io: { enabled: false }
  fsync: false

naming:
  charset: ascii
```

```
    alphabet: [lower, digit, dash]

  files:
    count: 120
    size: { min: 1MB, max: 20MB }
    extensions: [.dat, .bin, .log]
```

Full spec files will be written to `spec_files/RT-NN/` in the implementation step.

---

15. Implementation Steps (Ordered)

> Not starting until this plan is approved.

1. Copy `bryckclient-cli/` into `CloudCpReportTesting/bryckclient-cli/` (pre-fill login.json, cloud_ops.json template).
2. Create `config.json` with defaults from §4.
3. Write `spec_files/RT-01/` through `spec_files/RT-07/` YAML files.
4. Extend `report_engine.py` with live report parsing functions:
   - `parse_transfer_report_csv(path)` → list of row dicts
   - `parse_transfer_summary_txt(path)` → dict of counters
   - `unzip_report(zip_path, out_dir)` → path to extracted dir
5. Write `cases/rt_01_small_flat.py` through `cases/rt_10_round_trip.py` (plugin interface: `CASE_ID`, `DESCRIPTION`, `STEPS`, `run(context, out_dir)`).
6. Write `run_report_tests.py` with all CLI modes from §9.
7. Write cleanup functions (SSH + S3 subprocess).
8. Write cleanup assertion functions.
9. Validate end-to-end against a known good transfer (transfer_id 89 already on disk as `cloud_transfer_89/`).

---

16. Open Questions Before Implementation

- [ ] Does `datagen` produce a manifest of generated files (name + size)? If yes, we can diff against the report precisely. If no, we need an SSH `find + stat` to enumerate source files.
- [ ] Does the API at `POST /api/bcloud/transfer` accept `bryck_src` and `cloud_bucket` directly, or do those need to be set in `/api/bcloud/config` first (i.e., configure → initiate)?
- [ ] For the download transfer (RT-09/RT-10), does the API use the same `POST /api/bcloud/transfer` with reversed src/dst, or a different endpoint?
- [ ] Is there a `final_report.json` always present in the ZIP, or only when a specific config flag is set?
- [ ] Should the runner continue to the next case after a `TRANSFER_FAILED`, or stop the entire run?